ARCHON AI

AI-Powered System Design Assistant

Software Requirements Specification
& System Design Document

Submitted By:
Yuvraj Gupta, 2310991326

Table of Contents

S. No	Title	Page no.
1	Executive Summary	3
2	Stakeholders & Target Users	3 - 4
3	Functional Requirements	4 - 6
4	Non-Functional Requirements	7 - 9
5	System Architecture Design	9 - 12
6	Database Design	13 - 15
7	API Design	16 - 17
8	Frontend Architecture	18 - 19
9	Deployment Architecture	19 - 20
10	Risk Register	20
11	Appendix	21

1. Executive Summary

Archon AI is an intelligent system design assistant that transforms vague product ideas into production-ready system designs through AI-guided workflows. The platform bridges the gap between initial concept and technical specification by acting as a structured mediator between developers and AI systems.

> Core Value Proposition: Turn vague product ideas into production-ready system designs using AI-guided workflows — eliminating the friction of prompt engineering and missing architectural components.

1.1 Problem Being Solved

Most developers struggle to convert vague product ideas into structured system designs due to three core friction points:

- Lack of awareness of what structured information LLMs require to generate meaningful output
- Tendency to omit critical architectural components such as scalability strategy, database selection rationale, and API design
- Inconsistent and incomplete designs that cannot be directly used for production planning

1.2 Solution Overview

Archon AI provides a layered solution that extracts structured requirements through guided interaction, orchestrates a multi-step AI pipeline, and outputs comprehensive system design artifacts including architecture diagrams, database schemas, and API specifications.

2. Stakeholders & Target Users

2.1 Primary Users

User Type	Description	Primary Use Case
Full Stack Developers	Mid-level engineers building new products	Generate architecture blueprints quickly
Tech Leads / Architects	Senior engineers reviewing design options	Compare architectural patterns & cost estimates
Startup Founders	Non-technical founders with product ideas	Convert ideas into technical specs for dev teams
CS Students	Students learning system design	Practice and learn through guided examples

Table 2.1: Primary Users

2.2 Admin Users

- Platform Administrator — Manages users, monitors usage, and oversees system health
- Content Manager — Manages prompt templates and AI pipeline configurations

3. Functional Requirements

3.1 Authentication & Authorization

Requirement	Description	Acceptance Criteria
User Registration	Users register with name, email, and password	Registration succeeds; duplicate email returns error
User Login	Email/password login with JWT token issuance	Valid credentials return JWT; invalid return 401
Role-Based Access	Roles: User, Admin. Admin has elevated access	User cannot access admin endpoints; roles enforced
Token Refresh	JWT refresh mechanism to maintain sessions	Expired tokens refreshed without re-login
Password Reset	Reset via email with time-limited token	Reset link expires in 1 hour; new password accepted

Table 3.1: Authentication Methods

3.2 Project Management

Requirement	Description	Acceptance Criteria
Create Project	User creates a named project with description	Project saved and listed on dashboard
List Projects	Dashboard shows all user projects with metadata	Projects sorted by last updated; paginated
View Project	View full design output for a selected project	All design sections rendered correctly
Delete Project	User can delete a project and all its data	Project and associated designs removed from DB
Version History	Each design iteration saved as a new version	Versions listed and individually viewable

Table 3.2: Phases of Project

3.3 Guided Input Wizard

Requirement	Description	Acceptance Criteria
Raw Idea Input	Text area for user to input initial product idea	Input accepted; minimum 10 characters validated
AI-Driven Questioning	AI generates contextual follow-up questions	5-8 targeted questions generated within 10 seconds
Smart Suggestions	Suggest common answers as clickable chips	Suggestions rendered; user can select or type custom
Progress Indicator	Step-by-step wizard with clear progress	Current step highlighted; user can navigate back
Structured JSON Output	Wizard converts answers to structured JSON	Output JSON includes users, scale, features fields

Table 3.3: Project Input Flow

3.4 AI Design Generation Pipeline

The AI pipeline is the core processing engine of Archon AI, transforming structured input into comprehensive design artifacts through sequential processing stages.

Requirement	Description	Acceptance Criteria
Requirement Extraction	Parse wizard output into functional & non-functional requirements	Both requirement types populated in output
Architecture Generation	AI selects and justifies architecture (monolith vs microservices)	Includes rationale, components, and tech choices
API Design Generation	Auto-generate RESTful API endpoints with HTTP methods	Endpoints include method, path, request/response schema
Database Schema Generation	Generate DB schema based on domain model	Schema includes collections/tables, fields, and relationships
Caching Strategy	Suggest caching layer (Redis, CDN) based on scale	Strategy includes cache type, TTL, and invalidation logic
Diagram Generation	Generate Mermaid.js architecture diagram code	Valid Mermaid syntax; diagram renders without errors
Tech Stack Recommendation	AI recommends tech stack with justifications	Each recommendation includes rationale and alternatives

Table 3.4: AI Generation Flow

3.5 Chat-Based Iterative Refinement

Requirement	Description	Acceptance Criteria
Chat Panel	Persistent chat panel on design view page	Chat history preserved per project session
Incremental Updates	AI updates only the affected design section	Only modified sections re-render; others unchanged
Context Memory	AI retains context of previous design decisions	Follow-up requests reference prior design accurately
Version on Save	Each refinement saved as new version on user action	Save button creates version with timestamp

Table 3.5: Chat-Based Interaction Versioning

3.6 Export & Output

Requirement	Description	Acceptance Criteria
Markdown Export	Export design as Markdown file	Valid .md file with proper heading hierarchy
JSON Export	Export structured design data as JSON	Valid JSON matching design schema; downloadable

Table 3.6: Export Options

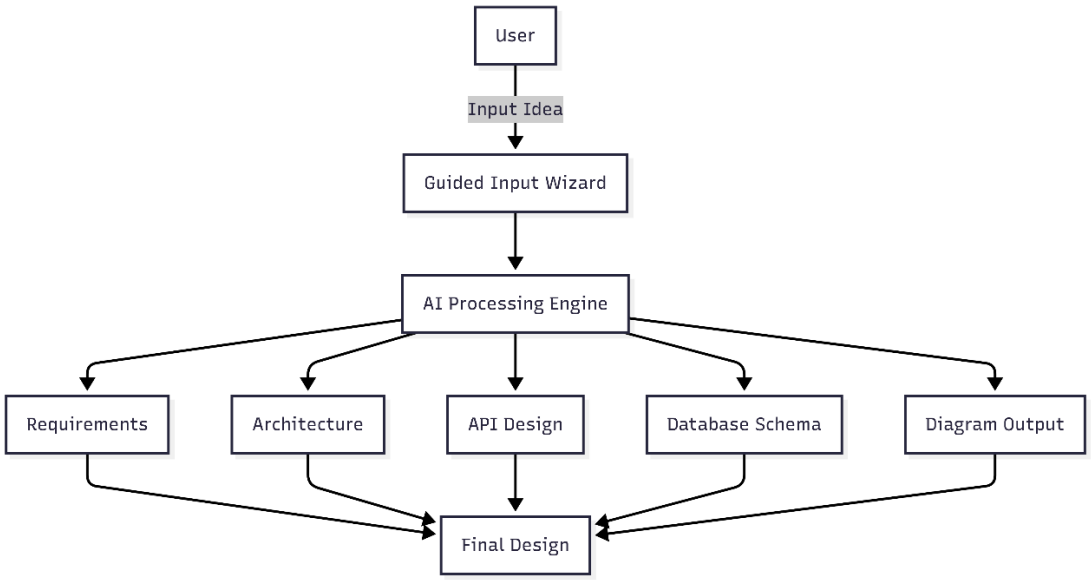

Figure 3.1: Data Flow Diagram

4. Non-Functional Requirements

4.1 Performance

Requirement	Target Metric	Measurement Method
AI Pipeline Response Time	Full design generation < 15 seconds	End-to-end API response time monitoring
Page Load Time	Initial load < 2 seconds (LCP)	Google Lighthouse; Core Web Vitals
API Response Time (Non-AI)	REST endpoints < 300ms (p95)	APM tool (Datadog / New Relic)
Wizard Step Transition	Each step transition < 100ms	React profiler measurements
Diagram Render Time	Mermaid diagrams render < 1 second	Browser performance.mark() measurements

Table 4.1: Performance Metrics

4.2 Scalability

Requirement	Target	Notes
Concurrent Users	Support 500 concurrent users (Phase 1)	Horizontal scaling via containerization
AI API Queue	Queue AI requests during peak load	Bull.js queue; graceful degradation
Database Scaling	MongoDB horizontal sharding for 1M+ projects	Atlas auto-scaling configuration
Stateless Backend	All backend services are stateless	Session state stored in DB; not in-process

Table 4.2: Scalability Feasability

4.3 Security

Requirement	Implementation	Standard
Authentication	JWT with RS256; 15-min access token lifetime	OWASP Authentication Guidelines
Data Encryption	TLS 1.2+ in transit; AES-256 at rest	NIST encryption standards
Input Validation	Server-side validation for all inputs	OWASP Input Validation Cheatsheet

Requirement	Implementation	Standard
Rate Limiting	100 req/min per user; 10 AI req/min per user	Express rate-limit middleware
AI Prompt Security	Sanitize inputs to prevent prompt injection	Strip control characters; validate length
RBAC Enforcement	Middleware validates role on every protected route	Role checked at route entry; fail-secure

Table 4.3: Security Options

4.4 Reliability & Availability

- System availability: 99.5% uptime SLA (excluding scheduled maintenance)
- AI service fallback: If primary AI provider is unavailable, fallback to secondary provider or queue request
- Database: MongoDB Atlas replica set with automatic failover
- Error handling: All API errors return structured JSON with error code and message
- Graceful degradation: Non-AI features (project listing, version history) remain operational during AI outage

4.5 Usability

- Wizard completes in under 5 minutes for a typical use case
- Mobile-responsive design supporting viewport widths from 375px to 4K
- Onboarding: New users guided via tooltips on first project creation
- Error messages are human-readable with suggested corrective actions

4.6 Maintainability

- Code coverage: Minimum 70% unit test coverage for backend services
- API versioning: All endpoints versioned under /api/v1/ prefix
- Logging: Structured JSON logs with correlation IDs for distributed tracing
- Documentation: OpenAPI 3.0 spec auto-generated from route annotations

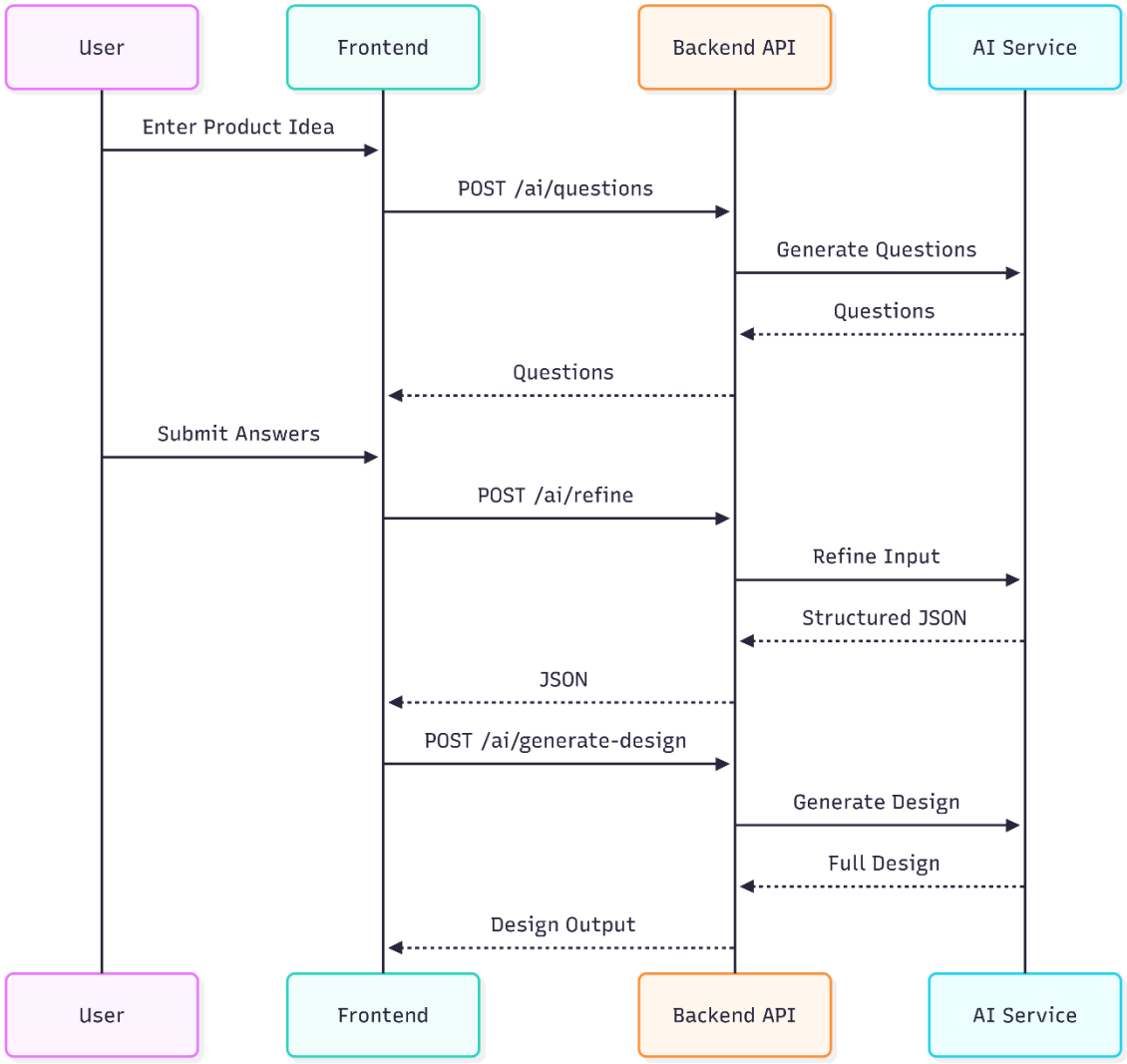

Figure 4.1: Sequence/User Interation Diagram

5. System Architecture Design

5.1 Architecture Overview

Archon AI follows a three-tier architecture with a dedicated AI orchestration layer. The system is designed for horizontal scalability while maintaining simplicity for the initial deployment phase.

Layer	Technology	Responsibilities
Presentation	React + Vite + Tailwind CSS	User interface, wizard flow, diagram rendering, state management

Layer	Technology	Responsibilities
API Gateway	Node.js + Express	Request routing, auth middleware, rate limiting, response transformation
Business Logic	Express Service Layer	Project CRUD, versioning logic, AI pipeline orchestration
AI Orchestration	Custom Pipeline	Multi-step prompt chaining, context management, response parsing
Data Persistence	MongoDB + Mongoose	User data, project designs, version history, prompt logs
Caching	Redis	Session tokens, AI response caching, rate limit counters
AI Provider	Claude API / OpenAI API / Ollama Local API	LLM inference for design generation and refinement

Table 5.1.1: System Architecture

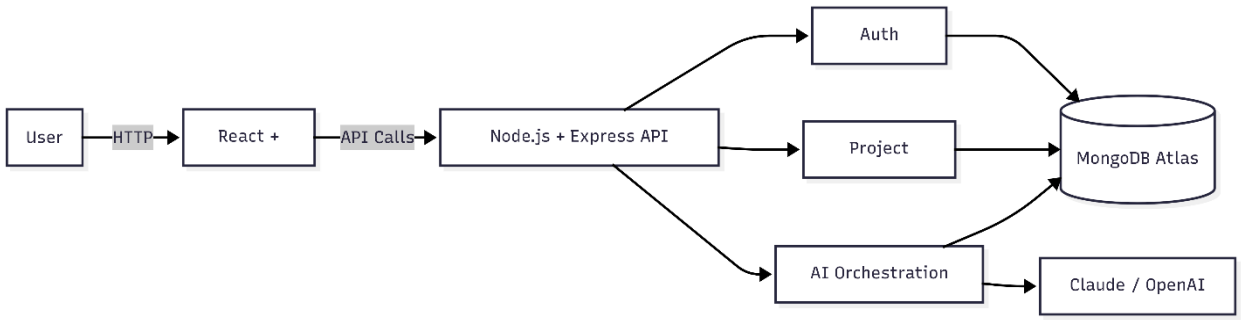

Figure 5.1: System Architecture Diagram

5.2 Architecture Decision: Monolith First

Decision: Start with a modular monolith architecture. Migrate to microservices when user base exceeds 10,000 active users or team grows beyond 5 engineers. This reduces operational complexity in the early stage while maintaining a clean module boundary for future extraction.

Rationale for Monolith-First:

- Simpler local development and debugging environment
- Reduced infrastructure cost (single Render/Railway deployment)
- Faster iteration cycles for MVP validation
- Module boundaries enforced via domain-driven folder structure enable future service extraction

5.3 AI Pipeline Architecture

The AI pipeline processes user input through five sequential stages, each responsible for a distinct transformation step:

Step	Stage	Input	Output
1	Raw Idea Capture	Unstructured text (e.g., 'build a food delivery app')	Stored raw input; triggers question generation
2	Prompt Refinement Engine	Raw idea + wizard answers	Structured JSON: {users, scale, features, constraints}
3	Requirement Structuring	Structured JSON	Functional + Non-functional requirements list
4	Design Generation	Requirements + tech constraints	Architecture, APIs, DB schema, tech stack
5	Diagram Generation	Architecture description	Mermaid.js diagram code (validated syntax)

Table 5.3.1: AI Pipeline Flow

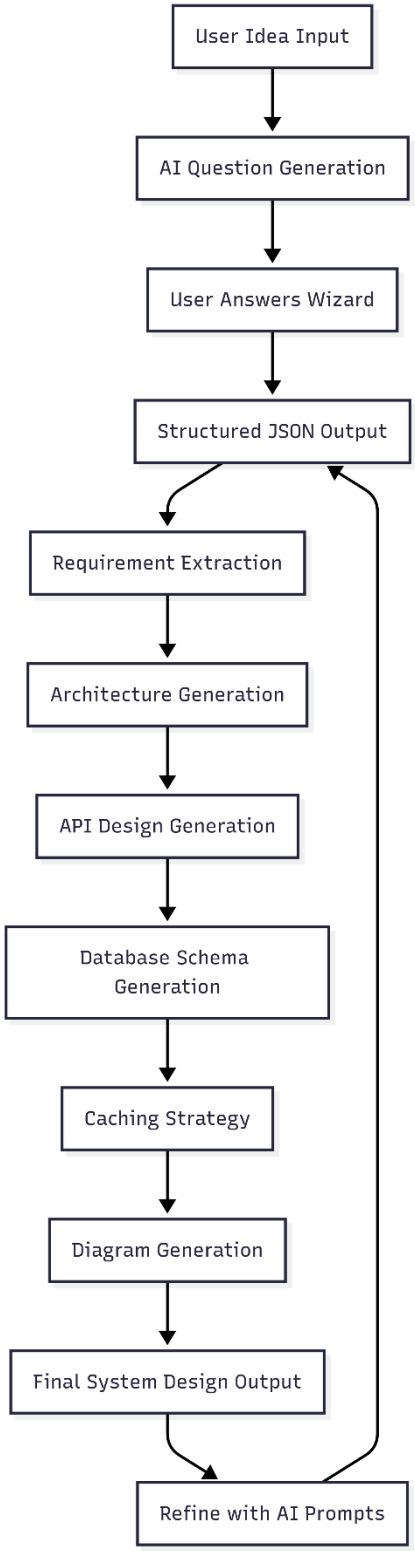

Figure 5.3.1: AI Pipeline Nodes

6. Database Design

6.1 Database Selection

MongoDB (via MongoDB Atlas) is selected as the primary database for the following reasons:

- Flexible schema accommodates evolving AI-generated output structures without migrations
- Native JSON document storage aligns with AI pipeline output format
- Atlas provides managed replica sets, auto-scaling, and built-in monitoring
- Rich query capabilities for version history and design diff operations

6.2 Collections & Schema Definitions

6.2.1 Users Collection

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
name	String	Yes	User's full name
email	String (unique)	Yes	User's email address — indexed for lookup
password	String (hashed)	Yes	bcrypt hashed password; never stored plain
role	Enum: user|admin	Yes	Default: 'user'; controls access permissions
createdAt	Date	Auto	Account creation timestamp
updatedAt	Date	Auto	Last account update timestamp
isActive	Boolean	Yes	Soft delete flag; default: true

Table 6.2.1: Users Table Schema

6.2.2 Projects Collection

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
userId	ObjectId (ref: Users)	Yes	Owner reference — indexed for dashboard queries
title	String	Yes	Project name (max 100 chars)
description	String	No	Optional project description (max 500 chars)
status	Enum: draft|active|archived	Yes	Default: 'draft'; controls visibility
latestVersionId	ObjectId (ref: Designs)	No	Quick reference to most recent design
createdAt	Date	Auto	Project creation timestamp

Field	Type	Required	Description
updatedAt	Date	Auto	Last modified timestamp

Table 6.2.2: Projects Table Schema

6.2.3 Designs Collection

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
projectId	ObjectId (ref: Projects)	Yes	Parent project reference
version	Integer	Yes	Auto-incremented version number
requirements.functional	Array<String>	No	List of functional requirements
requirements.nonFunctional	Array<String>	No	List of non-functional requirements
architecture	Object	No	Pattern, components, rationale, tech stack
apis	Array<Object>	No	API endpoints with method, path, description
dbSchema	Array<Object>	No	Collections/tables with fields and types
diagram	String (Mermaid)	No	Mermaid.js diagram code string
createdAt	Date	Auto	Version creation timestamp

Table 6.2.3: Design Table Schema

6.2.4 Prompts Collection

Field	Type	Required	Description
_id	ObjectId	Auto	MongoDB primary key
projectId	ObjectId (ref: Projects)	Yes	Parent project reference
designId	ObjectId (ref: Designs)	Yes	Associated design version
rawInput	String	Yes	Original user text before refinement
refinedInput	Object (JSON)	No	Structured JSON after wizard processing
aiResponse	String	No	Raw AI response before parsing
tokensUsed	Integer	No	Token consumption for cost tracking
createdAt	Date	Auto	Prompt creation timestamp

Table 6.2.4: Prompts Table Schema

6.3 Indexes

Critical indexes to ensure query performance:

- users: { email: 1 } — unique, for login lookup
- projects: { userId: 1, updatedAt: -1 } — for dashboard listing
- designs: { projectId: 1, version: -1 } — for version history
- prompts: { projectId: 1, createdAt: -1 } — for chat history

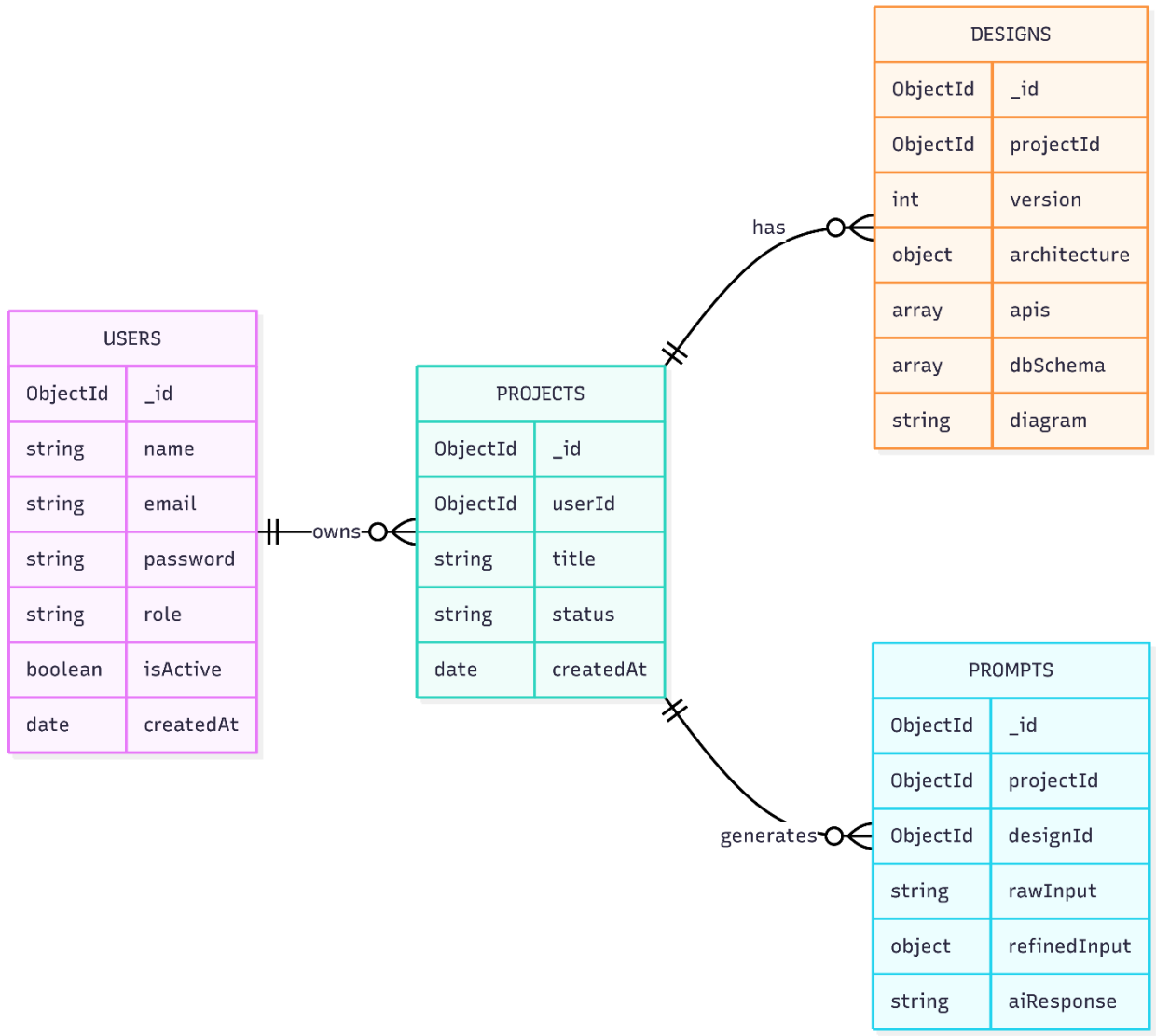

Figure 6.1: Database ER Diagram

7. API Design

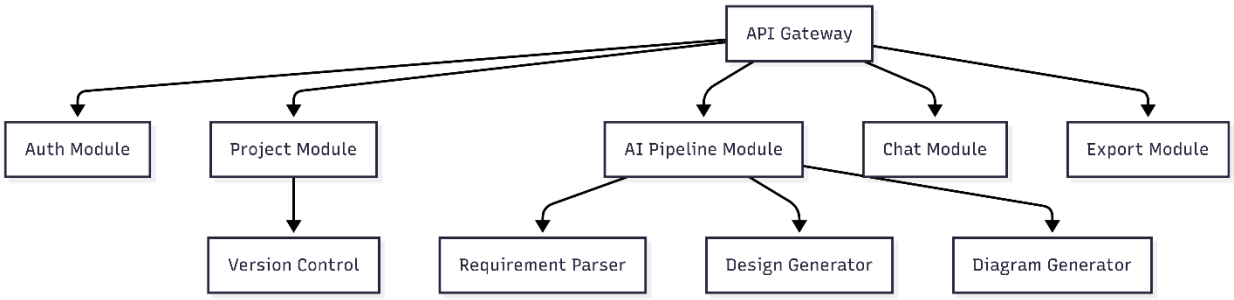

Figure 7.1: Backend Components Diagram

7.1 API Standards

- All endpoints prefixed with /api/
- Request/response format: application/json
- Authentication: Bearer token in Authorization header
- Error response format: { success: false, error: { code, message } }
- Success response format: { success: true, data: { ... } }

7.2 Authentication Endpoints

Method	Endpoint	Description	Auth Required	Status Codes
POST	/api/auth/register	Register new user account	No	201, 400, 409
POST	/api/auth/login	Login and receive JWT tokens	No	200, 400, 401
POST	/api/auth/refresh	Refresh access token	Refresh Token	200, 401
POST	/api/auth/logout	Invalidate refresh token	Yes	200, 401
POST	/api/auth/forgot-password	Send password reset email	No	200, 404

Table 7.2.1: Authentication API Endpoints

7.3 Project Endpoints

Method	Endpoint	Description	Auth Required	Status Codes
POST	/api/projects	Create new project	Yes	201, 400, 401

Method	Endpoint	Description	Auth Required	Status Codes
GET	/api/projects	List all user projects	Yes	200, 401
GET	/api/projects/:id	Get single project details	Yes	200, 401, 403, 404
PATCH	/api/projects/:id	Update project metadata	Yes	200, 400, 401, 403, 404
DELETE	/api/projects/:id	Delete project and all data	Yes	200, 401, 403, 404
GET	/api/projects/:id/versions	Get version history for project	Yes	200, 401, 403, 404

Table 7.3.1: Project API Endpoints

7.4 AI Pipeline Endpoints

Method	Endpoint	Description	Auth Required	Status Codes
POST	/api/ai/questions	Generate wizard questions from raw idea	Yes	200, 400, 401, 429
POST	/api/ai/refine	Refine wizard answers to structured JSON	Yes	200, 400, 401, 429
POST	/api/ai/generate-design	Generate full system design	Yes	200, 400, 401, 429
POST	/api/ai/update-design	Update design via chat instruction	Yes	200, 400, 401, 429
POST	/api/ai/generate-diagram	Generate Mermaid diagram from architecture	Yes	200, 400, 401, 429

Table 7.4.1: AI Pipeline API Endpoints

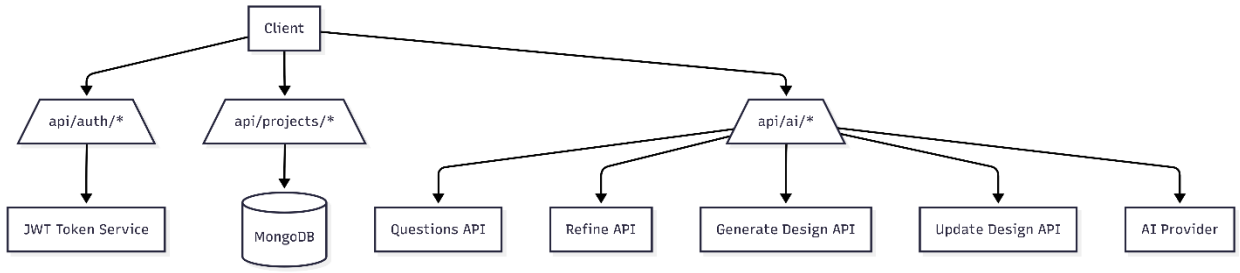

Figure 7.2: API Interaction Diagram

8. Frontend Architecture

8.1 Tech Stack

Technology	Version	Purpose
React	18.x	UI component library and rendering engine
Vite	5.x	Build tool and dev server with HMR
Tailwind CSS	3.x	Utility-first styling framework
React Flow	11.x	Interactive architecture diagram rendering
Mermaid.js	10.x	Auto-generated diagram code rendering
React Query	5.x	Server state management and caching
Zustand	4.x	Client-side global state management
React Router	6.x	Client-side routing and navigation
Axios	1.x	HTTP client with interceptors for auth tokens

Table 8.1: Tech Stack

8.2 Page Inventory

Page	Route	Key Components
Landing Page	/	Hero section, feature highlights, CTA, testimonials
Login / Register	/auth/login, /auth/register	Auth forms, validation, JWT storage
Dashboard	/dashboard	Project cards, search/filter, create project CTA
Create Project Wizard	/projects/new	Multi-step wizard, smart suggestions, progress bar
Design View	/projects/:id	Tabbed view: Requirements, Architecture, APIs, Diagram
Chat Refinement Panel	Slide-over on /projects/:id	Chat input, message history, diff highlighting
Version History	/projects/:id/versions	Version timeline, comparison view
Admin Dashboard	/admin	User management, usage stats, system health

Table 8.2: Pages in the App

8.3 State Management Strategy

State is divided into two categories based on lifecycle:

State Type	Tool	Examples
Server State (remote data)	React Query	Projects list, design data, version history
Global Client State	Zustand	Auth user, wizard step, active project
Local Component State	useState / useReducer	Form inputs, UI toggles, modal visibility
URL State	React Router	Active tab, selected version ID

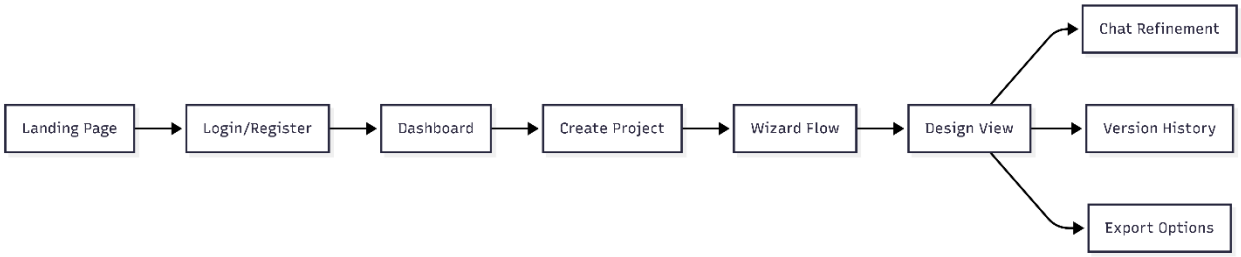

Figure 8.1: Frontend Flow Diagram

9. Deployment Architecture

9.1 Infrastructure Overview

Component	Platform	Configuration
Frontend	Vercel	Auto-deploy on main branch; edge CDN; preview deployments for PRs
Backend API	Render / Railway	Web service; auto-deploy; 512MB RAM (starter)
Database	MongoDB Atlas (M10)	Replica set; 10GB storage; automated backups daily
AI	Gemini API / Ollama	Light-weight API; Fast-Response

Table 9.1: Authentication API Endpoints

9.2 CI/CD Pipeline

Automated pipeline triggered on every push to main and develop branches:

1. Code push to GitHub triggers CI workflow
2. Run linting (ESLint) and unit tests (Jest / Vitest)
3. Build Docker image and run integration tests

4. Deploy to staging environment automatically
5. Manual approval gate for production deployment
6. Post-deployment health check; auto-rollback on failure

9.3 Environment Configuration

Three environments maintained:

- Development — Local developer machines; uses local MongoDB and mocked AI responses
- Staging — Mirrors production infrastructure; connected to real AI API with test keys
- Production — Live environment; full monitoring, alerting, and backup enabled

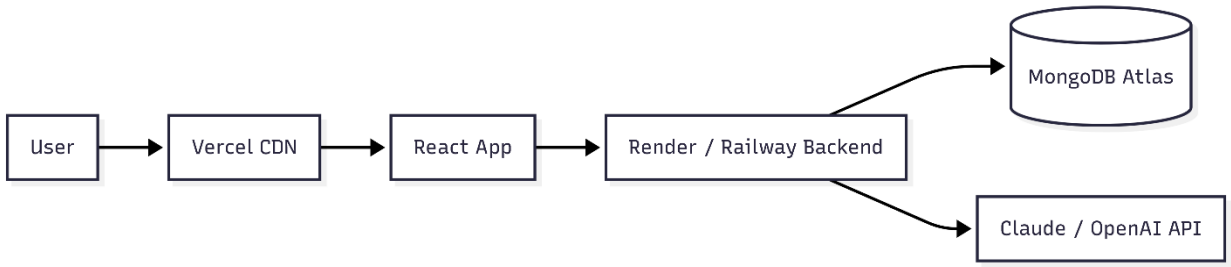

Figure 9.1: Deployment Diagram

10. Risk Register

Risk	Likelihood	Impact	Mitigation
AI API rate limit / downtime	Medium	High	Request queueing; fallback to secondary provider; graceful error UI
AI response inconsistency (hallucinations)	High	Medium	Output validation schema; retry with refined prompt; human review flag
Prompt injection attacks	Medium	High	Input sanitization; prompt boundary enforcement; output filtering
MongoDB Atlas cost overrun	Low	Medium	TTL indexes for prompt logs; Atlas cost alerts; data archival policy
JWT token theft	Low	High	Short-lived access tokens; HttpOnly cookie for refresh token; IP binding
Slow AI pipeline UX	High	Medium	Streaming responses; step-by-step progress indicator; background processing

Table 10.1: Possible Risks

11. Appendix

11.1 Glossary

Term	Definition
AI Pipeline	Sequence of LLM API calls that transform user input into structured design output
Prompt Refinement Engine	The first AI stage that converts raw user text into structured JSON via guided questioning
Mermaid.js	JavaScript library that renders text-based diagram definitions as visual diagrams
JWT	JSON Web Token — a compact, URL-safe token format used for authentication
RBAC	Role-Based Access Control — permission system tied to user roles
LLM	Large Language Model — AI model trained on text for natural language processing tasks
DXA	Document XML Attribute units — unit used in OOXML (Word) documents; 1440 DXA = 1 inch